4/22/2026

Warehouse Management System - AI

Submitted By:

Name: Jatin Kevlani

USN No: 2410111703

Name: Saumya Adani

USN No: 2410111002

Division: CSE-B Batch: B1

Submitted To:

Ms. Nidhi Acharya

Academic Year:2025-26

Jatin Kevlani, 2410111703
Saumya Adani, 2410111002

CASE STUDY: Warehouse Management System - AI

- **Abstract**

WMS-AI is a full-stack Warehouse Management System built to manage inventory, suppliers, orders, sales, alerts, analytics, and AI-based recommendations from a single dashboard. It uses a Spring Boot backend with a browser interface to improve warehouse operations through automation, monitoring, reporting, and decision support. The system is also designed as an academic case study for object-oriented programming. It demonstrates concepts like abstraction, inheritance, polymorphism, encapsulation, interfaces, constructors, exceptions, collections, recursion, and file handling within real business workflows, making it both practically useful and academically meaningful for students and developers learning software design principles.

- **Functional Requirement**
  - User authentication with JWT-based login and role-oriented access for Admin and Staff.
  - Product management with add, update, delete, search, stock-in, and stock-out operations.
  - Category and supplier management for organizing inventory and vendor records.
  - Purchase order lifecycle management with statuses such as pending, sent, received, and cancelled.
  - Sales transaction recording with automatic stock deduction and audit logging.
  - Low-stock and overstock alert generation with duplicate suppression and email notification support.
  - AI-powered recommendation endpoints for restocking advice, product bundling, and demand forecasting.
  - Analytics dashboard for inventory value, sales trends, alert count, and business insights.
  - File export and import support for CSV-based product and sales data handling.
  - Coverage of OOPJ practical concepts through real Java classes, utilities, and workflow modules.

- **UML Diagram:**

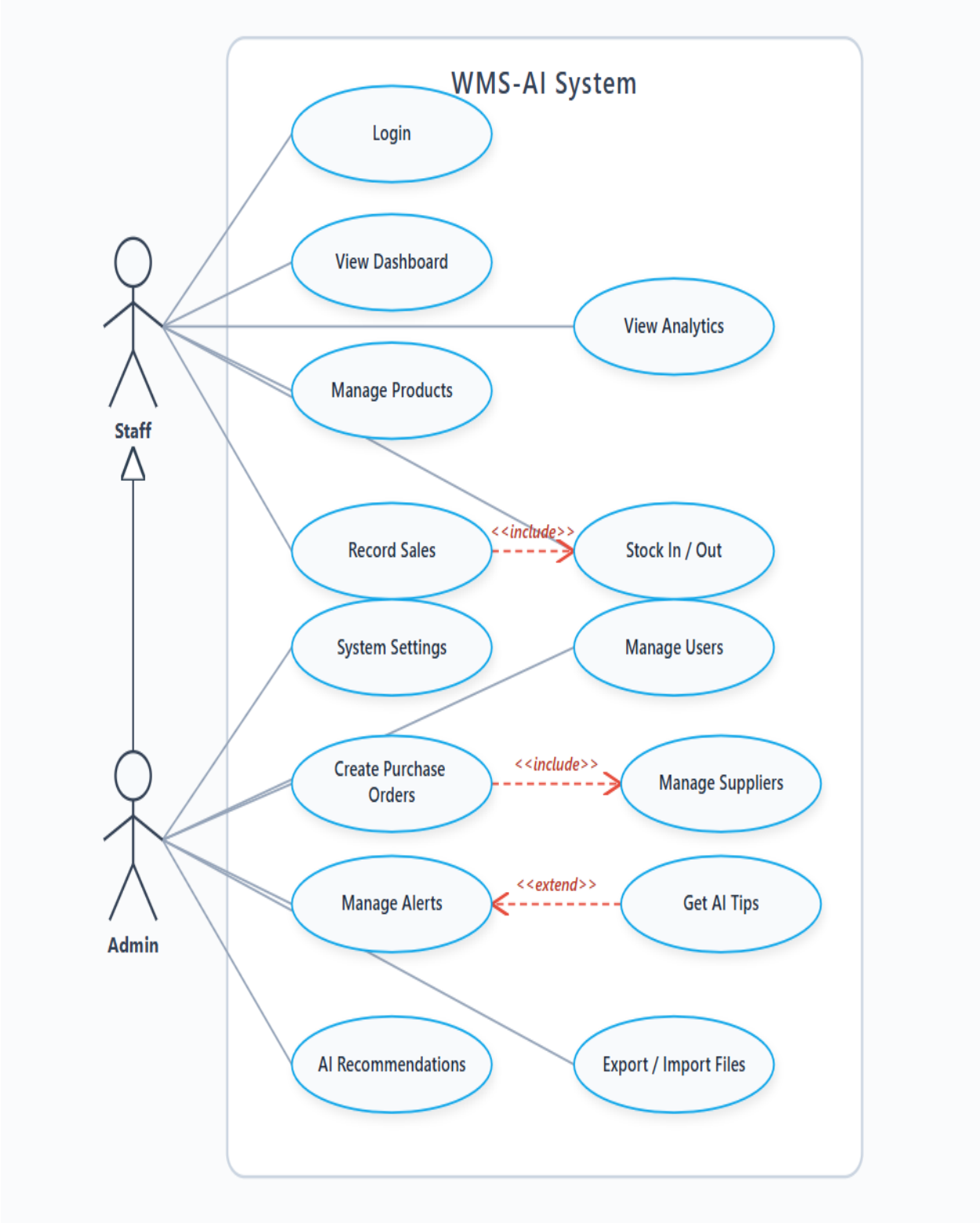

Fig 1 – Use Case Diagram

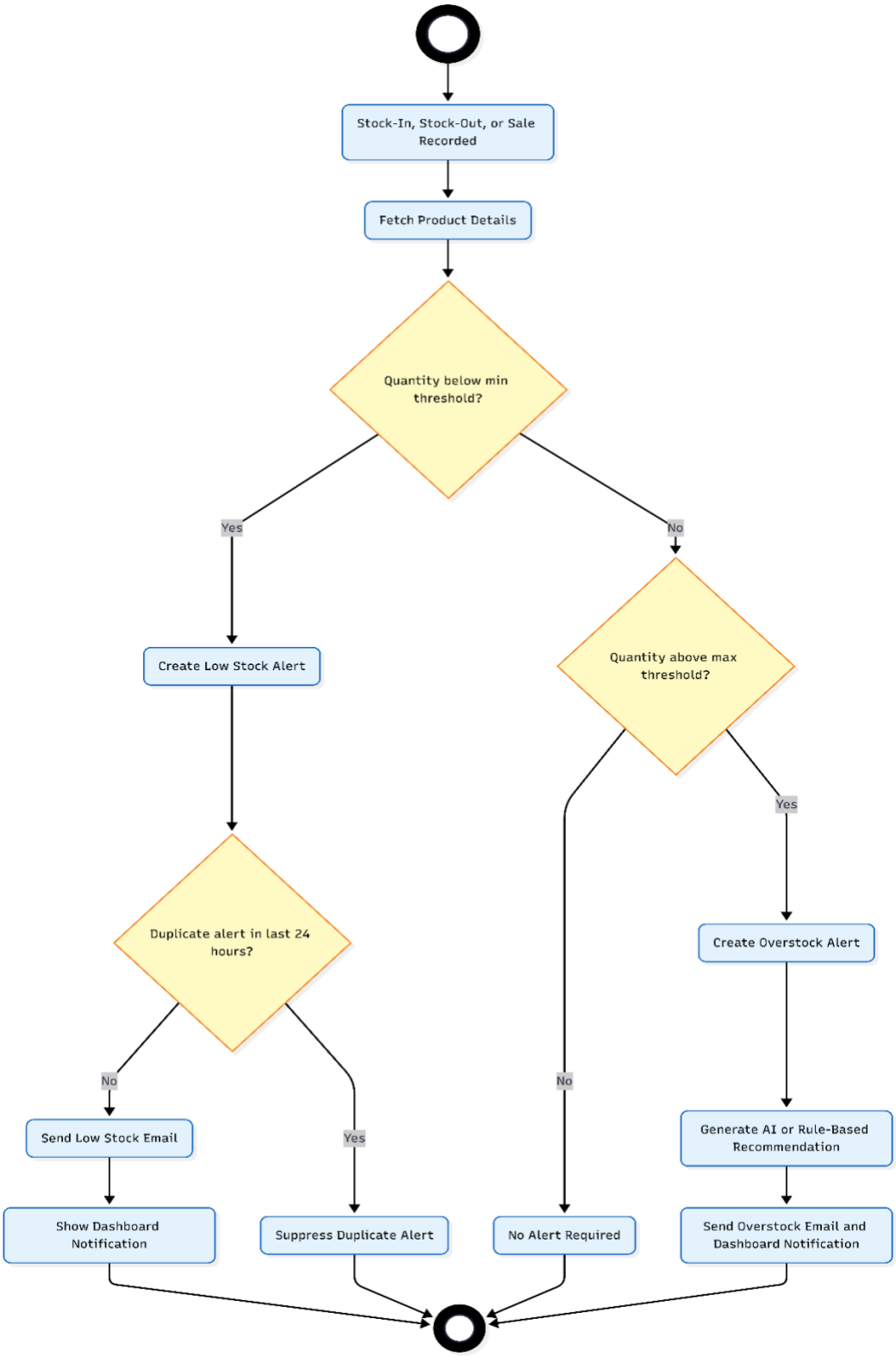

Fig 2 – Activity Diagram

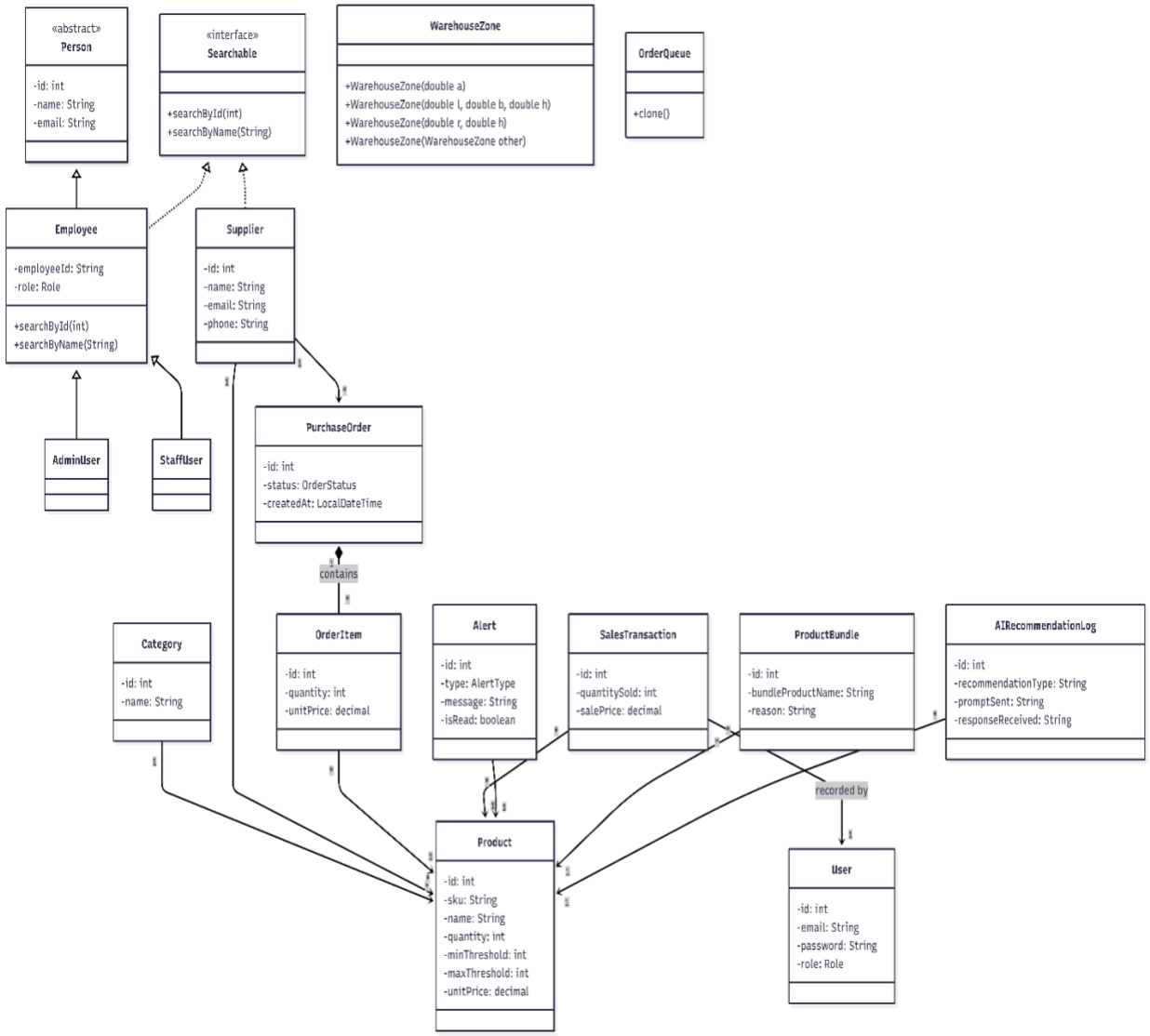

Fig 3 – Class Diagram

- **Structure or Flow Diagram**

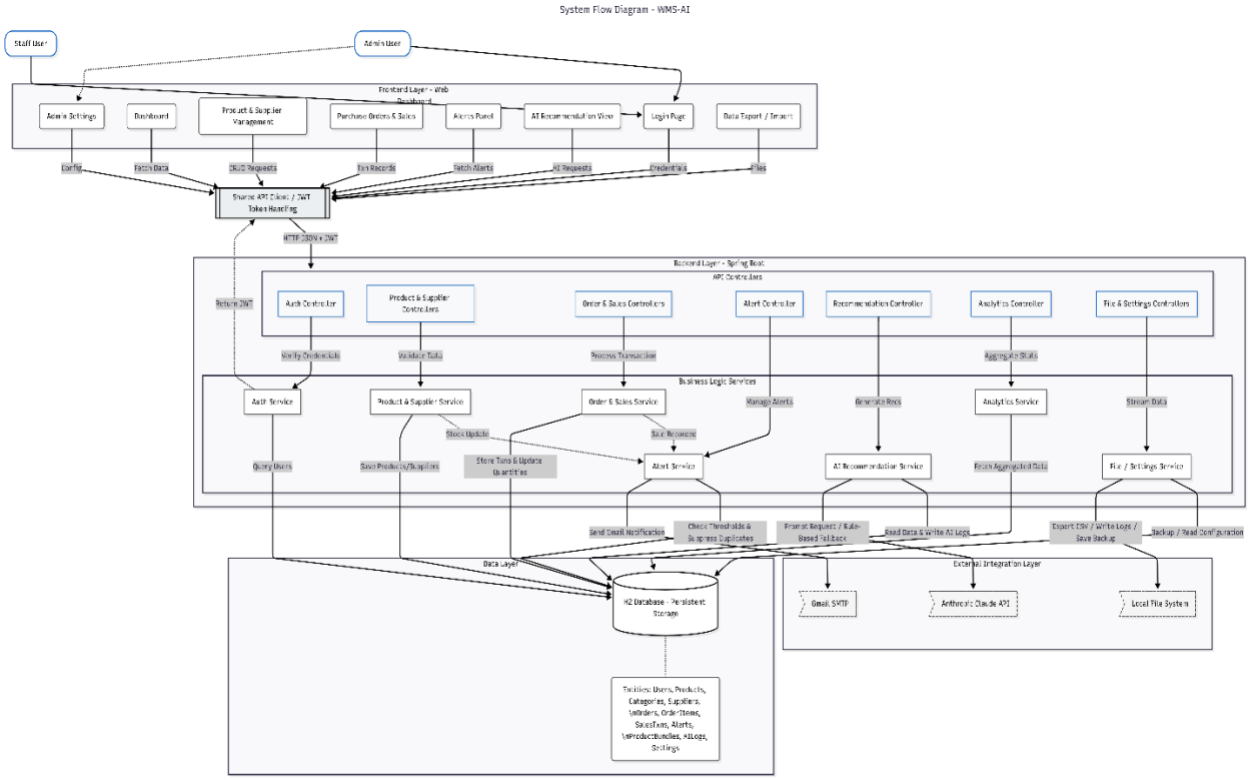

Fig 4 – Flow Diagram

- **Git Hub Link:** https://github.com/JatinKevlani/WMS-AI/
- **Input:**

Abstraction: Person is modeled as an abstract base class for employee hierarchy.

```
public abstract class Person {
    private String name;
    private String email;
    private String phone;

    public Person(String name, String email, String phone) {
        this.name = name;
        this.email = email;
        this.phone = phone;
    }

    public abstract String getDisplayInfo();
}
```

File Location: backend/src/main/java/com/wmsai/oopj/Person.java (lines 9-33)

Inheritance: Employee extends Person, while AdminUser and StaffUser extend Employee.

```
public class Employee extends Person implements Searchable {
    public Employee(String name, String email, String phone, int employeeId, String
department) {
        super(name, email, phone);
        this.employeeId = employeeId;
        this.department = department;
    }
}
```

File Location: backend/src/main/java/com/wmsai/oopj/Employee.java (lines 7-17)

```
public class AdminUser extends Employee {
    public AdminUser(String name, String email, String phone, int employeeId, String
accessLevel) {
        super(name, email, phone, employeeId, "Administration");
        this.accessLevel = accessLevel;
    }
}
```

File Location: backend/src/main/java/com/wmsai/oopj/AdminUser.java (lines 7-15)

Polymorphism: shared behaviors are exposed through parent types and overridden methods such as searchById and searchByName.

```
@Override
public String getDisplayInfo() {
    return "Employee[ID=" + employeeId + ", Name=" + getName() + ", Dept=" + department +
"]";
}

@Override
public Object searchById(int id) {
    if (this.employeeId == id) {
        return this;
    }
    return null;
}

@Override
public Object searchByName(String name) {
    if (getName() != null && getName().toLowerCase().contains(name.toLowerCase())) {
        return this;
    }
    return null;
}
```

File Location: backend/src/main/java/com/wmsai/oopj/Employee.java (lines 25-47)

Encapsulation: entity and model data are stored through private fields with controlled accessors.

```
private String name;
private String email;
```

```
private String phone;

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }
```

File Location: backend/src/main/java/com/wmsai/oopj/Person.java (lines 10-30)

Interface: Searchable is implemented by Supplier and Employee-oriented classes.

```
public interface Searchable {
    Object searchById(int id);
    Object searchByName(String name);
}
```

File Location: backend/src/main/java/com/wmsai/oopj/Searchable.java (lines 7-12)

```
public class Employee extends Person implements Searchable {
    private int employeeId;
    private String department;
}
```

File Location: backend/src/main/java/com/wmsai/oopj/Employee.java (lines 7-9)

Constructor overloading and copy constructor: WarehouseZone supports multiple constructors including a copy constructor.

```
public WarehouseZone(double side) {
    this.type = "Cube";
    this.volume = side * side * side;
    this.zoneName = "Zone-Cube-" + (int) side;
}

public WarehouseZone(double length, double breadth, double height) {
    this.type = "Cuboid";
    this.volume = length * breadth * height;
    this.zoneName = "Zone-Cuboid-" + (int) length + "x" + (int) breadth + "x" + (int)
height;
}

public WarehouseZone(double radius, double height, boolean isCylinder) {
    this.type = "Cylinder";
    this.volume = Math.PI * radius * radius * height;
    this.zoneName = "Zone-Silo-R" + (int) radius;
}

public WarehouseZone(WarehouseZone other) {
    this.type = other.type;
    this.volume = other.volume;
```

```
    this.zoneName = other.zoneName + "-COPY";
}
```

File Location: backend/src/main/java/com/wmsai/oopj/WarehouseZone.java (lines 18-43)

Method overloading: utility and search operations demonstrate overloaded methods.

```
public List<Product> search(String query) {
    return productRepository.searchByNameOrSku(query);
}

public List<Product> search(int id) {
    return List.of(getProductById(id));
}

public List<Product> search(String name, String category) {
    return productRepository.searchByNameAndCategory(name, category);
}
```

File Location: backend/src/main/java/com/wmsai/service/ProductService.java (lines 114-125)

```
public double calculateArea(double length, double width) {
    return length * width;
}

public double calculateArea(double radius) {
    return Math.PI * radius * radius;
}
```

File Location: backend/src/main/java/com/wmsai/oopj/AreaCalculator.java (lines 10-17)

Collections: ArrayList, PriorityQueue, and repository-managed collections are used across services and OOP utility classes.

```
public class NotificationQueue extends ArrayList<Object> {
    public void addNotification(Object item) {
        this.add(item);
    }

    public String getSummary() {
        long strings = stream().filter(o -> o instanceof String).count();
        long dates = stream().filter(o -> o instanceof LocalDate).count();
        long zones = stream().filter(o -> o instanceof WarehouseZone).count();
        long others = size() - strings - dates - zones;
        return "NotificationQueue[total=" + size() + ", strings=" + strings +
                ", dates=" + dates + ", zones=" + zones + ", others=" + others + "]";
    }
}
```

File Location: backend/src/main/java/com/wmsai/oopj/NotificationQueue.java (lines 10-33)

```
public class OrderQueue extends PriorityQueue<PurchaseOrder> implements Cloneable {
    @Override
    public OrderQueue clone() {
```

```
        OrderQueue cloned = new OrderQueue();
        cloned.addAll(this);
        return cloned;
    }
}
```

File Location: backend/src/main/java/com/wmsai/oopj/OrderQueue.java (lines 11-25)

```
try (PrintWriter writer = new PrintWriter(new FileWriter(filePath))) {

writer.println("ID,SKU,Name,Category,Supplier,Price,Quantity,MinThreshold,MaxThreshold,Last
SaleDate,CreatedAt");

    List<Product> products = productRepository.findAll();
    for (Product p : products) {
        writer.printf("%d,%s,%s,%s,%s,%.2f,%d,%d,%d,%s,%s%n",
                p.getId(),
                escapeCsv(p.getSku()),
                escapeCsv(p.getName()),
                p.getCategory() != null ? escapeCsv(p.getCategory().getName()) : "",
                p.getSupplier() != null ? escapeCsv(p.getSupplier().getName()) : "",
                p.getUnitPrice() != null ? p.getUnitPrice() : 0,
                p.getQuantity(),
                p.getMinThreshold(),
                p.getMaxThreshold(),
                p.getLastSaleDate() != null ? p.getLastSaleDate() : "",
                p.getCreatedAt());
    }
}
```

File Location: backend/src/main/java/com/wmsai/service/FileExportService.java (lines 45-61)

Exception handling: InsufficientStockException, DuplicateSKUException, InvalidSKUException, and ProductNotFoundException capture domain failures.

```
public class InsufficientStockException extends Exception {
    public InsufficientStockException(String message) {
        super(message);
    }
}
```

File Location: backend/src/main/java/com/wmsai/exception/InsufficientStockException.java (lines 7-10)

```
@ExceptionHandler(InsufficientStockException.class)
public ResponseEntity<Map<String, Object>>
handleInsufficientStock(InsufficientStockException ex) {
    return buildError(HttpStatus.BAD_REQUEST, ex.getMessage());
}

@ExceptionHandler(DuplicateSKUException.class)
public ResponseEntity<Map<String, Object>> handleDuplicateSKU(DuplicateSKUException ex) {
    return buildError(HttpStatus.CONFLICT, ex.getMessage());
}

@ExceptionHandler(ProductNotFoundException.class)
```

```
public ResponseEntity<Map<String, Object>> handleProductNotFound(ProductNotFoundException
ex) {
    return buildError(HttpStatus.NOT_FOUND, ex.getMessage());
}

@ExceptionHandler(InvalidSKUException.class)
public ResponseEntity<Map<String, Object>> handleInvalidSKU(InvalidSKUException ex) {
    return buildError(HttpStatus.BAD_REQUEST, ex.getMessage());
}
```

File Location: backend/src/main/java/com/wmsai/exception/GlobalExceptionHandler.java (lines 35-52)

Recursion and file handling: RecursiveUtils, InventoryLogWriter, TagDeduplicator, and export/import services satisfy file-I/O practical coverage.

```
public static int findSmallest(int[] arr, int index) {
    if (arr == null || arr.length == 0) {
        throw new IllegalArgumentException("Array must not be null or empty");
    }

    if (index == 0) {
        return arr[0];
    }

    int smallestOfRest = findSmallest(arr, index - 1);
    return Math.min(arr[index], smallestOfRest);
}
```

File Location: backend/src/main/java/com/wmsai/oopj/RecursiveUtils.java (lines 17-30)

```
public static int[] writeRandomStockLog(String filePath) throws IOException {
    Files.createDirectories(Path.of(filePath).getParent());

    Random random = new Random();
    int[] values = new int[COUNT];

    try (PrintWriter writer = new PrintWriter(new FileWriter(filePath))) {
        for (int i = 0; i < COUNT; i++) {
            values[i] = random.nextInt(1000) + 1;
            writer.print(values[i]);
            if (i < COUNT - 1) writer.print(" ");
        }
    }

    return values;
}
```

File Location: backend/src/main/java/com/wmsai/oopj/InventoryLogWriter.java (lines 23-38)

```
public static List<String> readAndDeduplicate(String filePath) throws IOException {
    Set<String> uniqueWords = new LinkedHashSet<>();

    try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
        String line;
        while ((line = reader.readLine()) != null) {
```

```
            String[] words = line.trim().split("\s+");
            for (String word : words) {
                if (!word.isEmpty()) {
                    uniqueWords.add(word.toLowerCase());
                }
            }
        }
    }

    List<String> sorted = new ArrayList<>(uniqueWords);
    sorted.sort(Collections.reverseOrder());
    return sorted;
}
```

File Location: backend/src/main/java/com/wmsai/oopj/TagDeduplicator.java (lines 21-38)

```
public String exportProductsCsv() throws IOException {
    String dir = basePath + "/exports";
    Files.createDirectories(Path.of(dir));

    String filename = "products_" + timestamp() + ".csv";
    String filePath = dir + "/" + filename;

    try (PrintWriter writer = new PrintWriter(new FileWriter(filePath))) {

writer.println("ID,SKU,Name,Category,Supplier,Price,Quantity,MinThreshold,MaxThreshold,Last
SaleDate,CreatedAt");
        List<Product> products = productRepository.findAll();
    }
}
```

File Location: backend/src/main/java/com/wmsai/service/FileExportService.java (lines 38-49)

```
public int importProductsCsv(MultipartFile file) throws IOException {
    List<String> errors = new ArrayList<>();
    int successCount = 0;

    try (BufferedReader reader = new BufferedReader(new
InputStreamReader(file.getInputStream()))) {
        String header = reader.readLine(); // skip header
        if (header == null) throw new IllegalArgumentException("Empty CSV file");

        String line;
        int lineNum = 1;
        while ((line = reader.readLine()) != null) {
            lineNum++;
        }
    }
}
```

File Location: backend/src/main/java/com/wmsai/service/FileImportService.java (lines 35-45)

- **Output:**

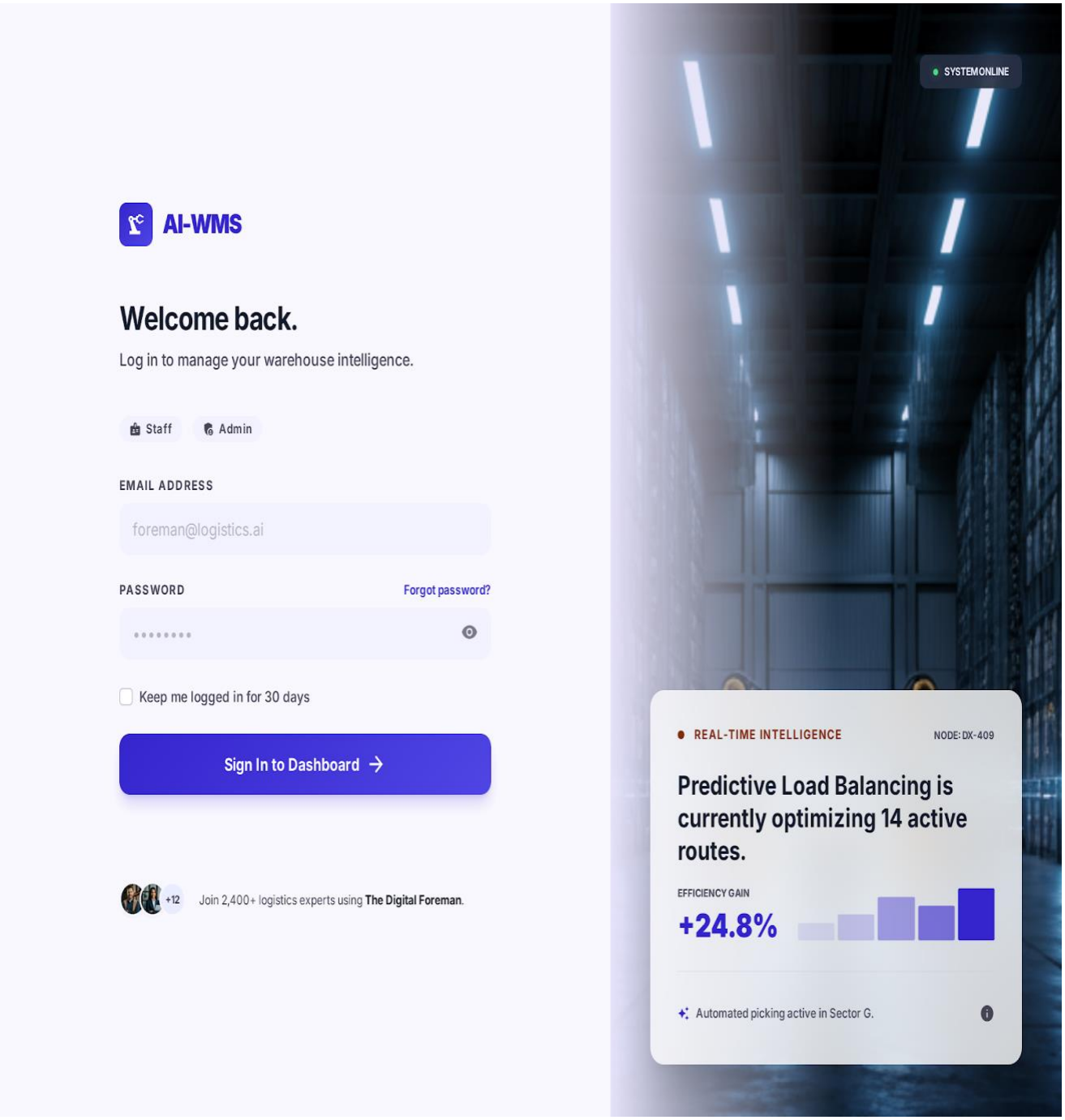

Fig 5 – Login Page

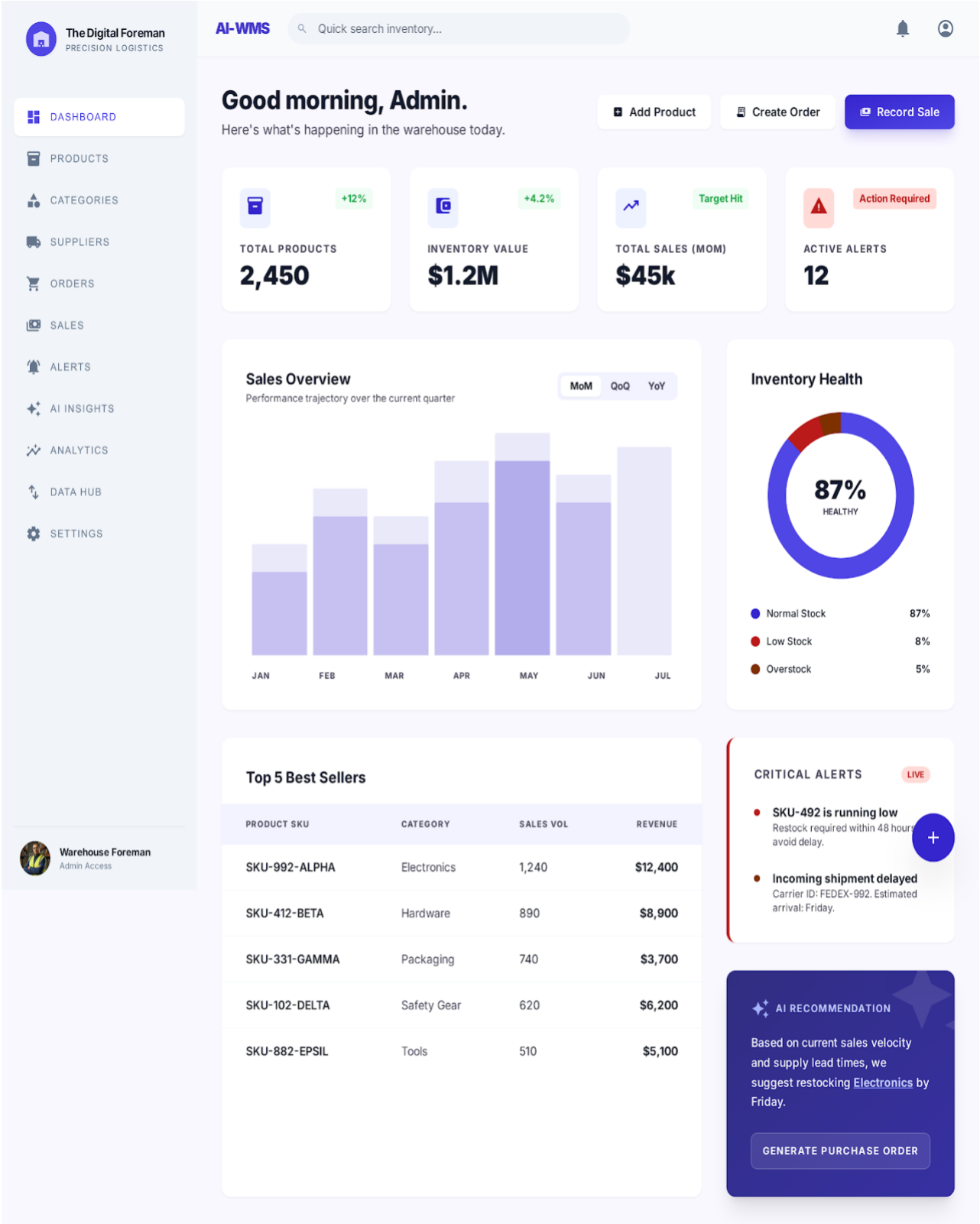

Fig 6 – Admin Dashboard

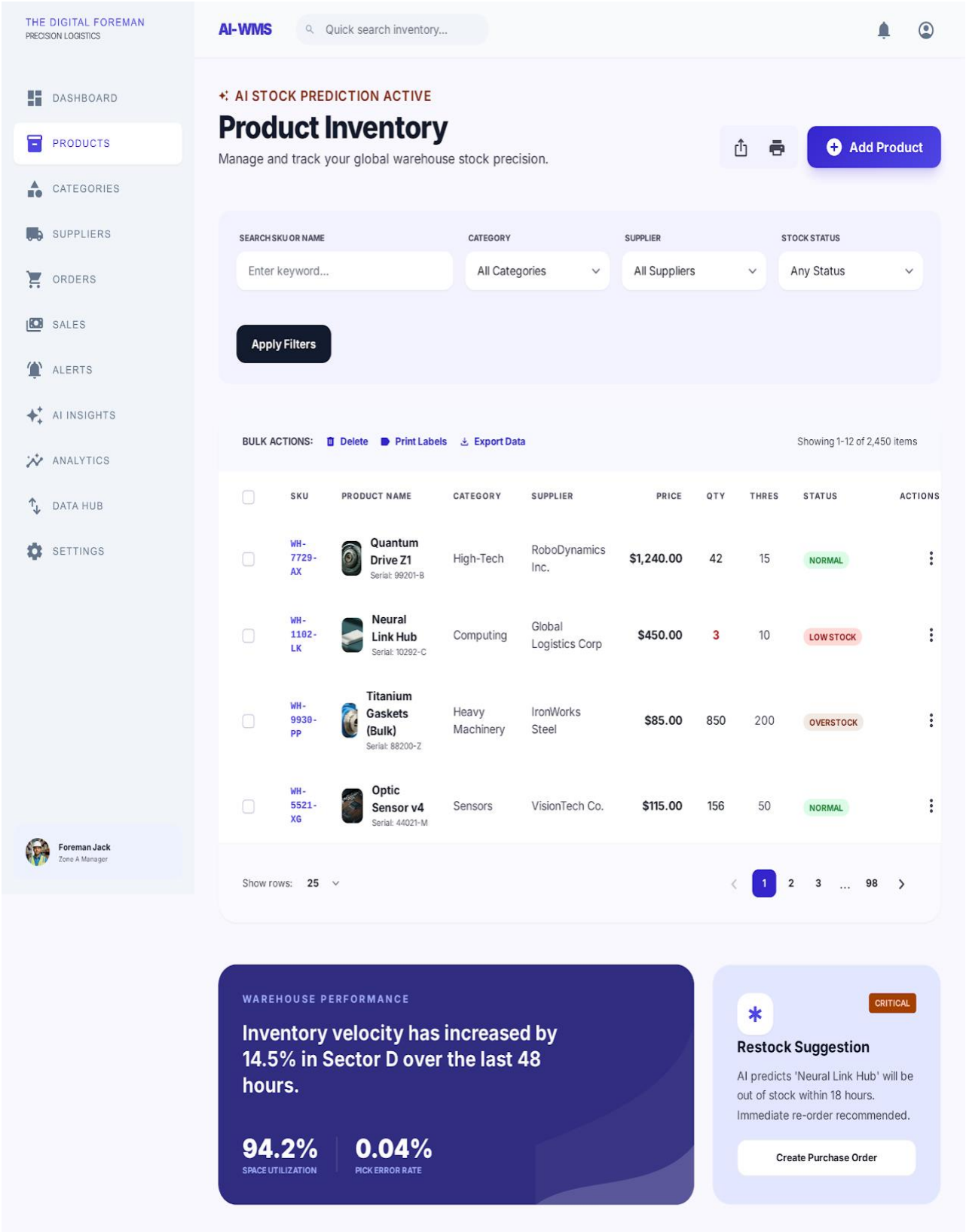

Fig 7 – Product List Module

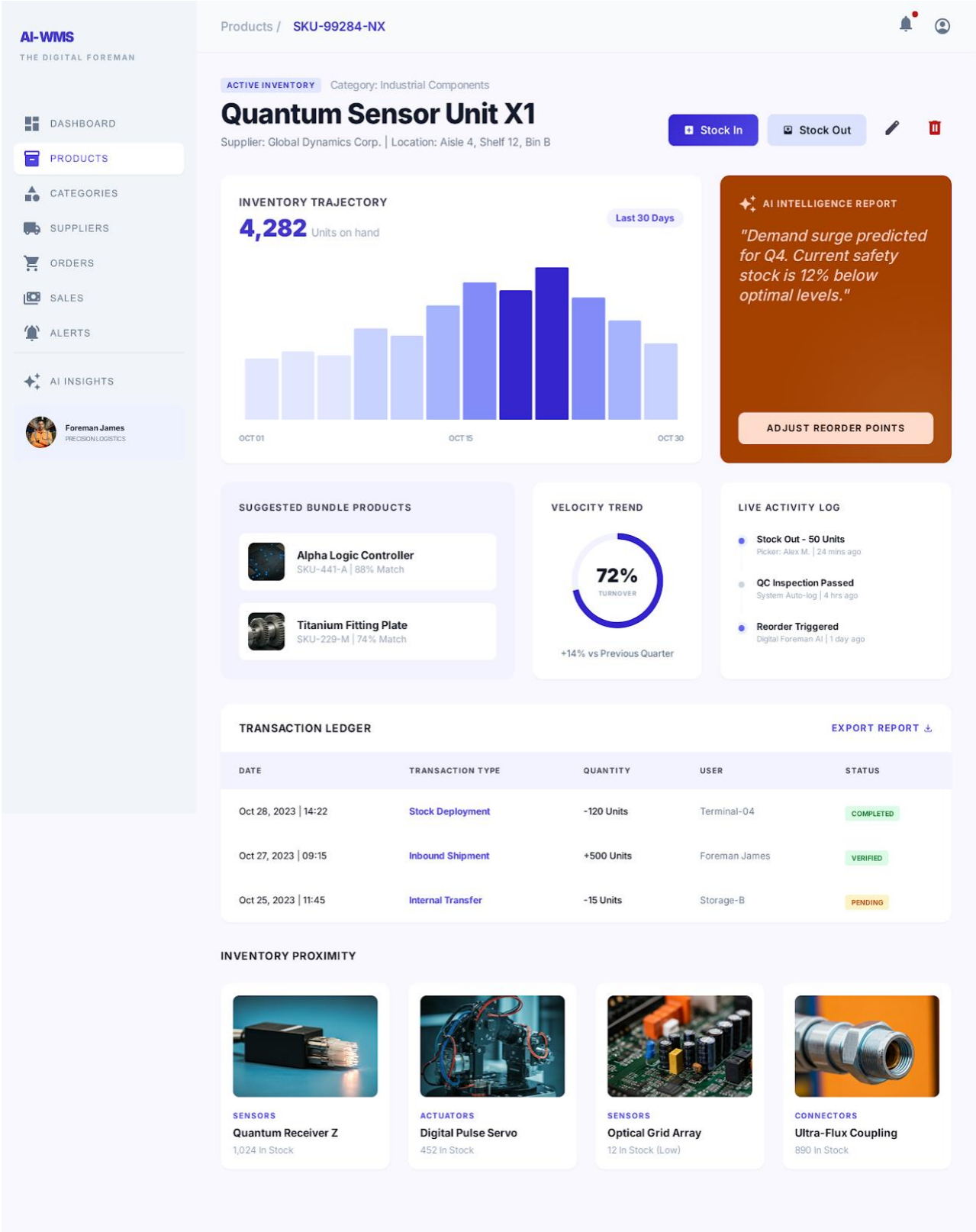

Fig 8 – Product Details Module

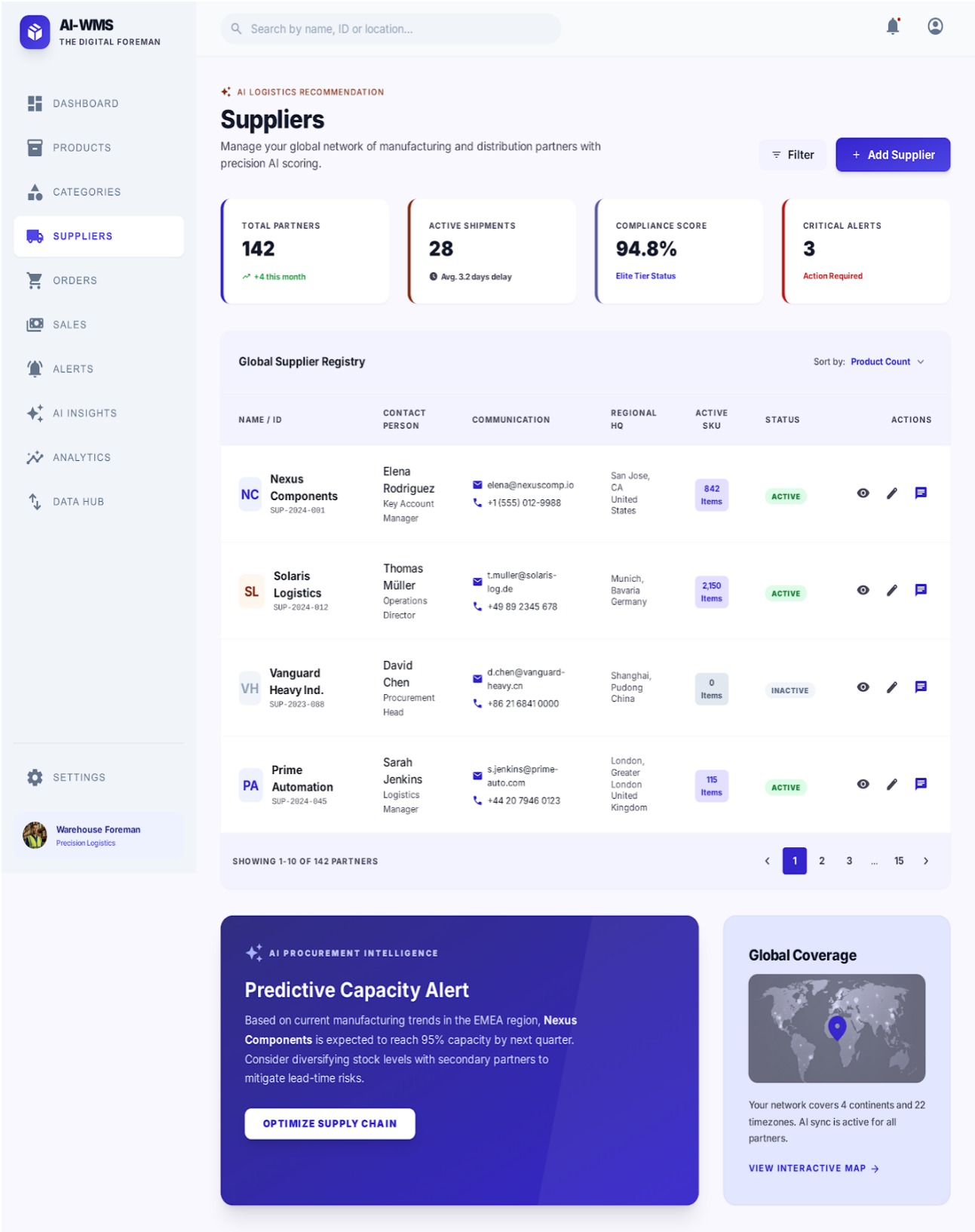

Fig 9 – Supplier Management Module

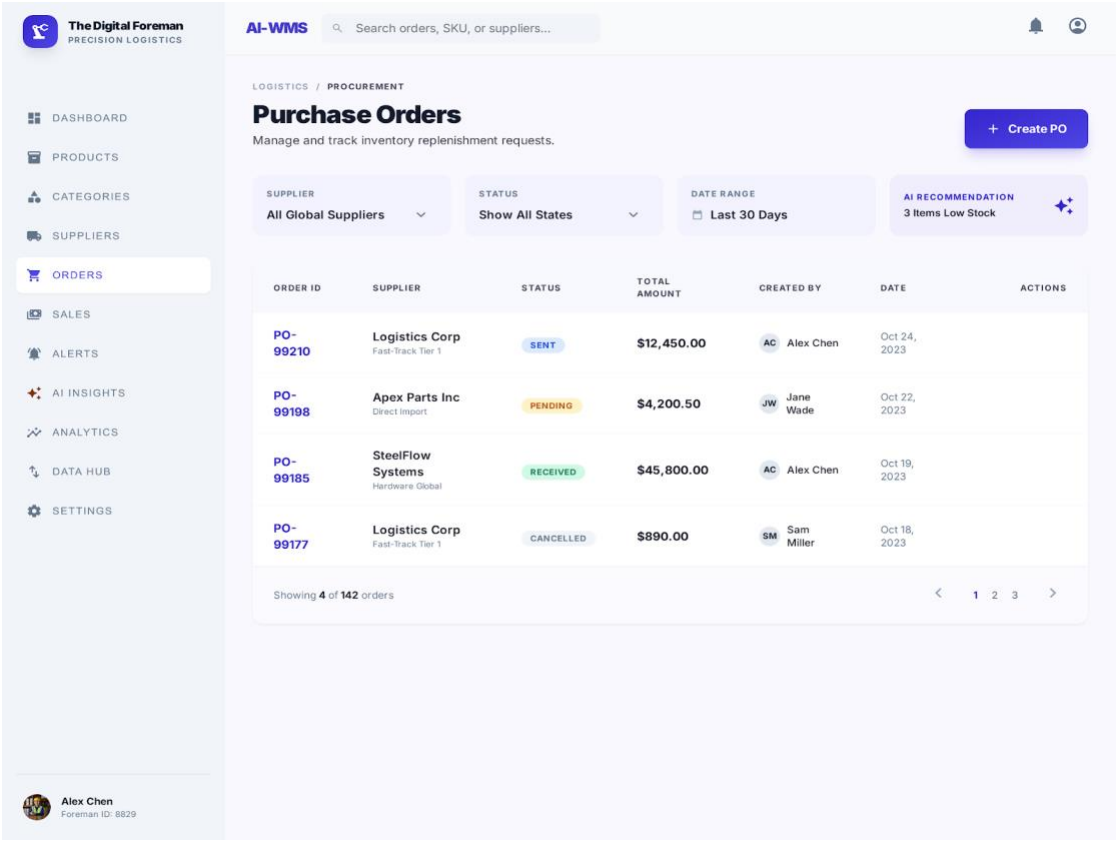

Fig 10 - Purchase Order Module

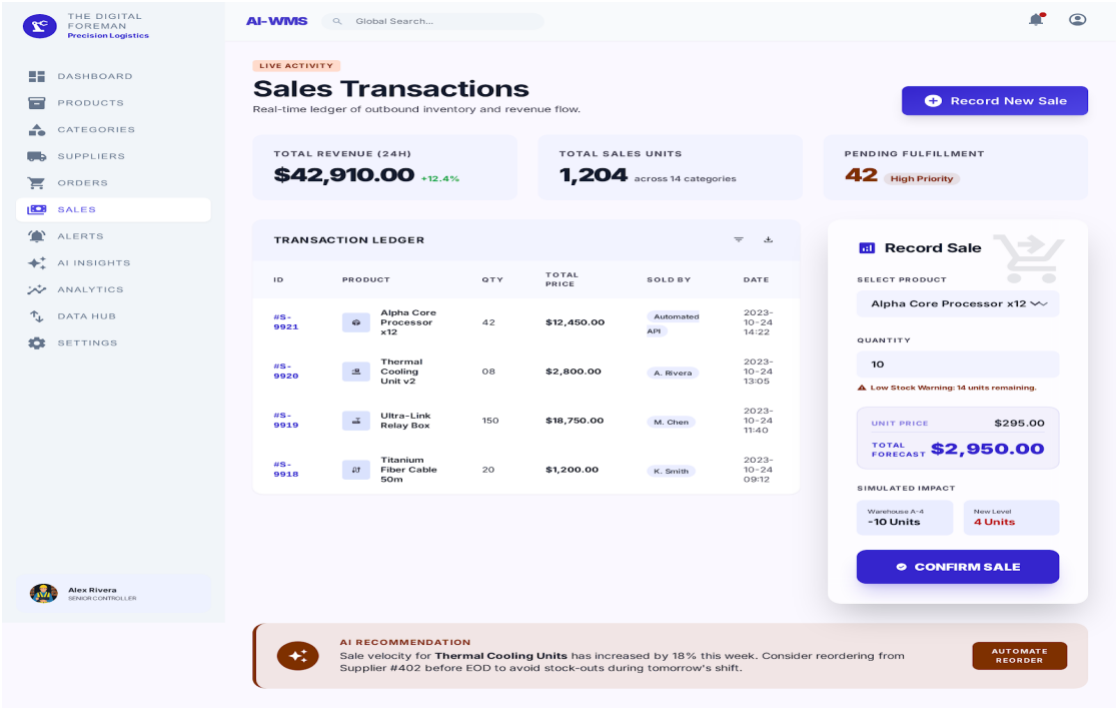

Fig 11 - Sales Transactions Module

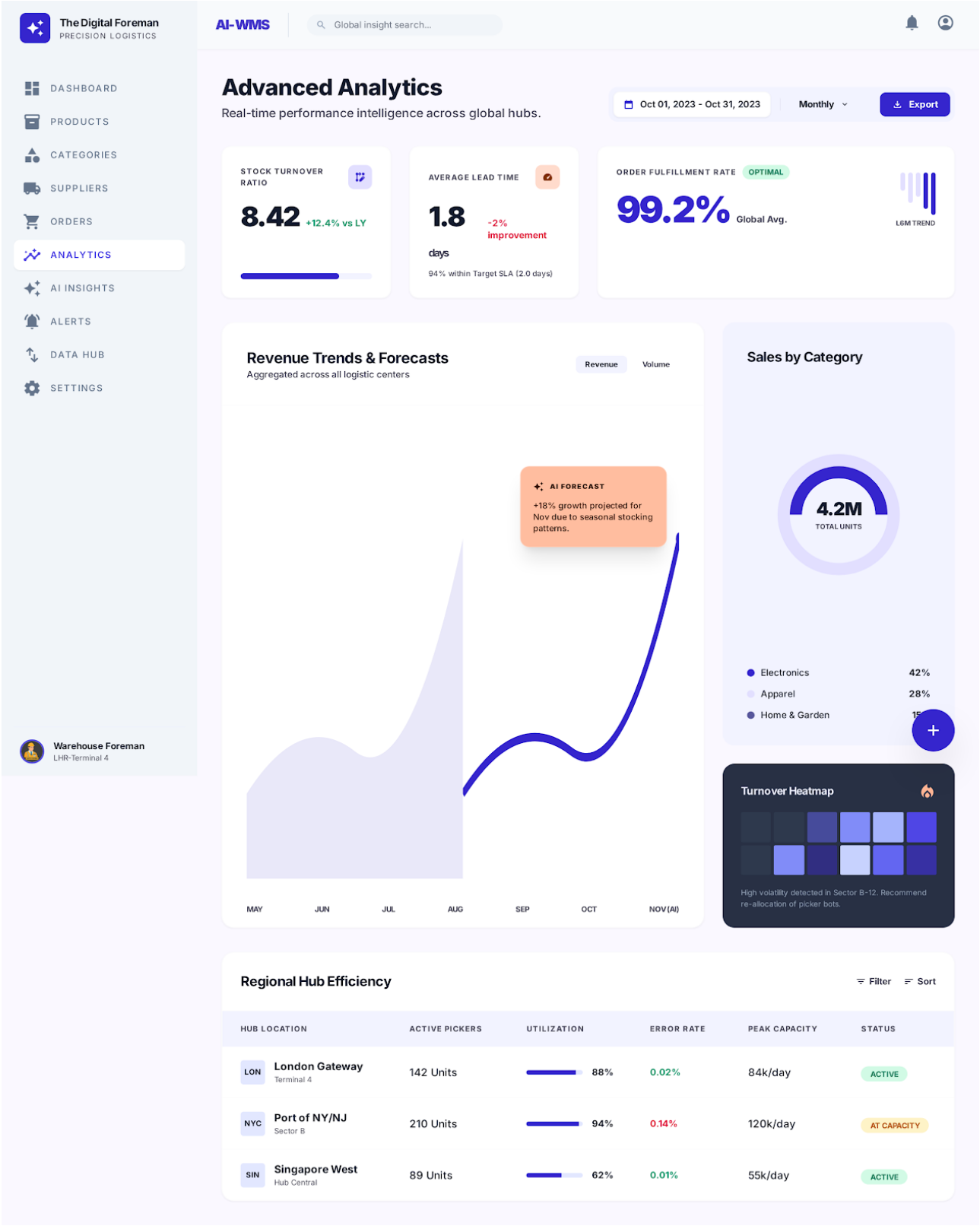

Fig 12 - Analytics Dashboard

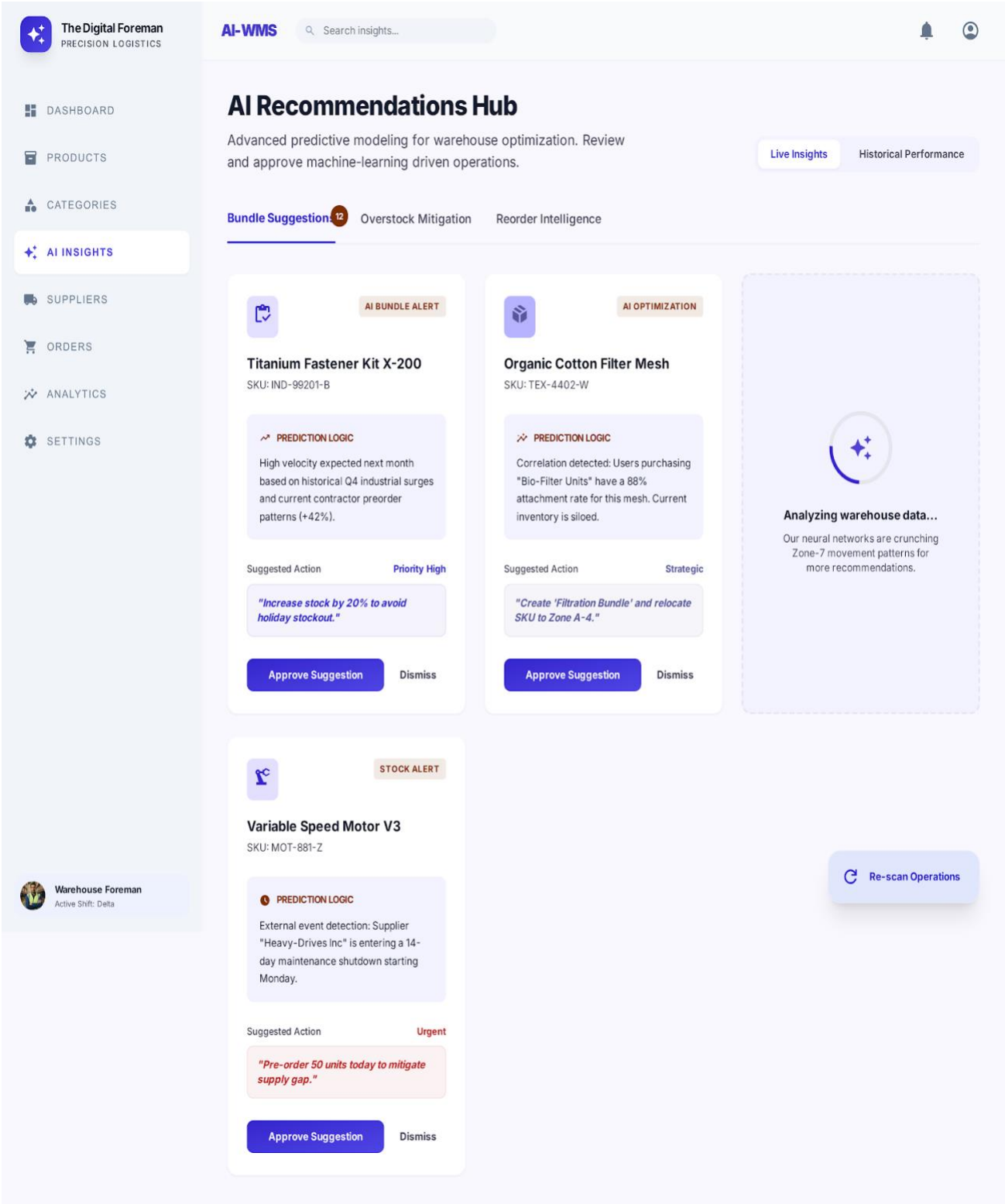

Fig 13 - AI Recommendations Module

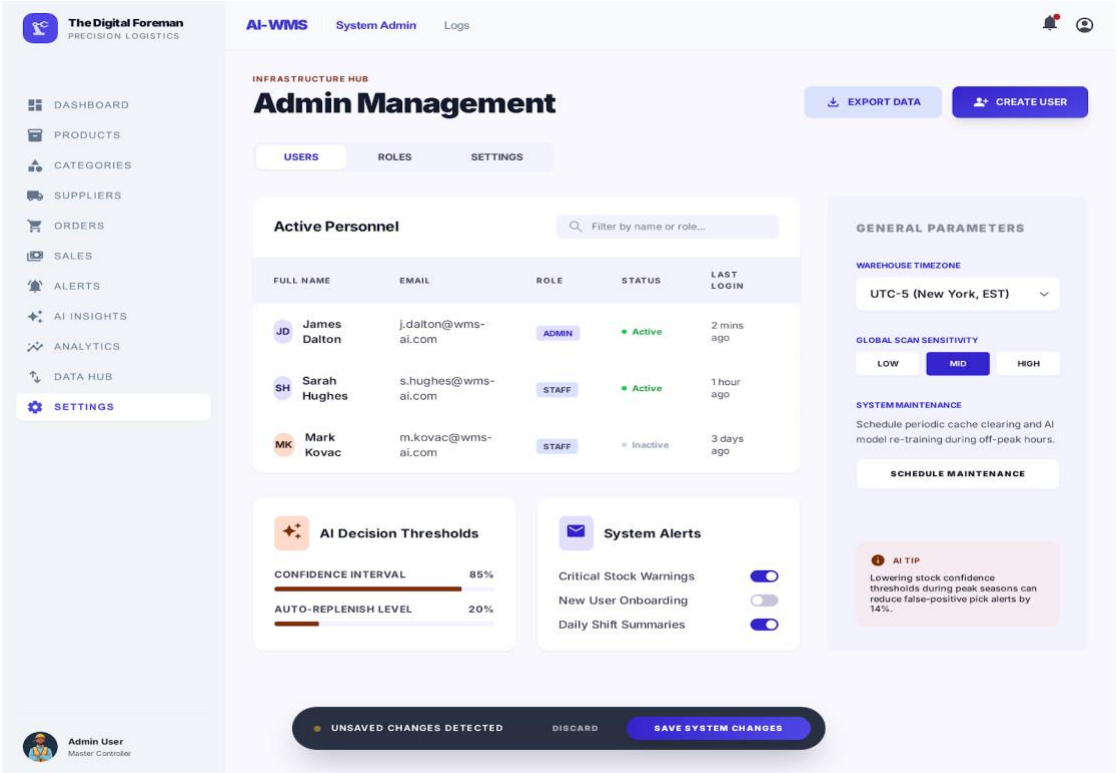

Fig 14 - Admin Settings Module

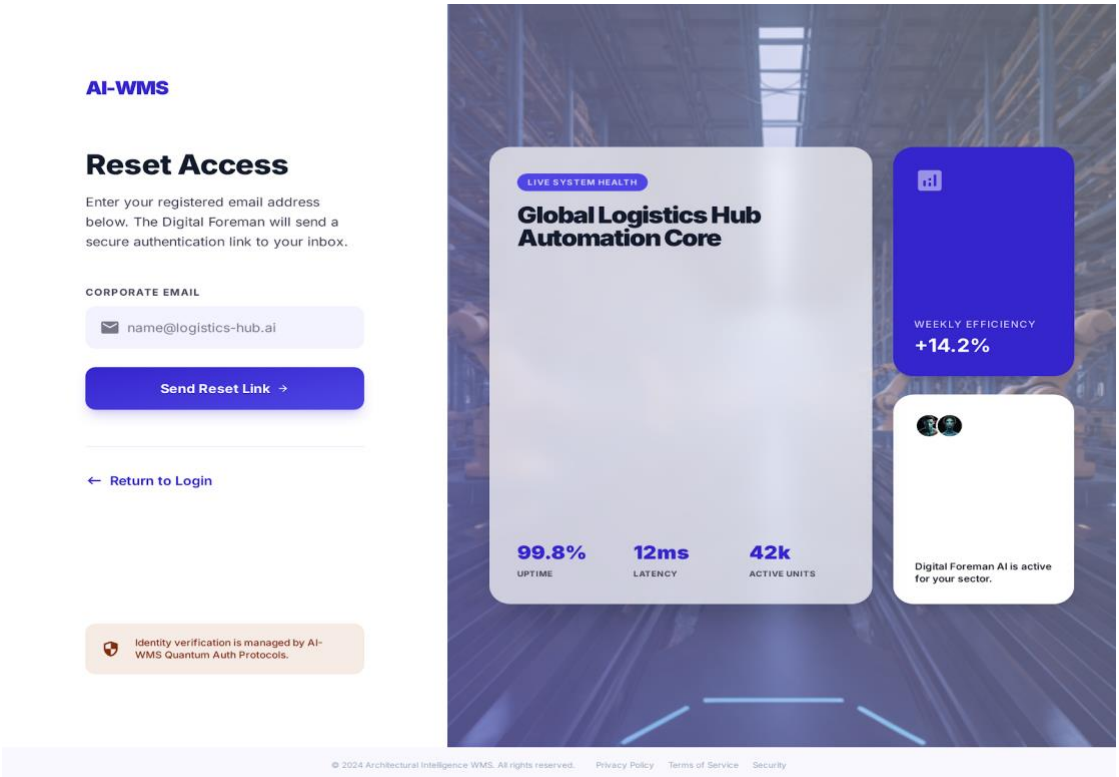

Fig 15 - Forgot Password Screen

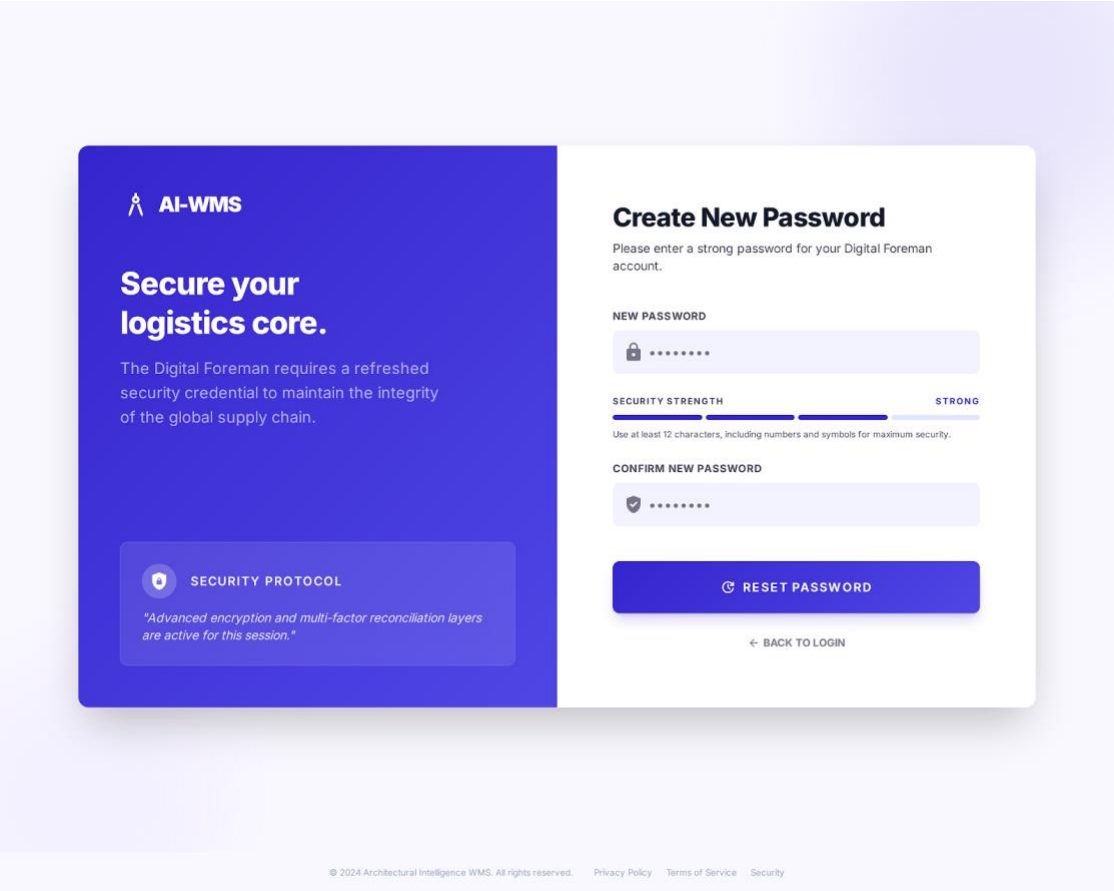

Fig 16 - Reset Password Screen

- ## Conclusion

  WMS-AI successfully demonstrates how warehouse operations can be digitized and improved through a secure, modular, and intelligent software solution. The project handles inventory, suppliers, orders, sales, alerts, analytics, and AI assistance inside one coherent system. From an academic perspective, it also proves that OOPJ concepts can be embedded inside a realistic enterprise application rather than treated as isolated lab exercises.

- ## Future Enhancement
  - Deploy the project with a production database such as PostgreSQL or MySQL instead of local development storage.
  - Add barcode or QR scanning support for faster warehouse operations.
  - Introduce multi-warehouse and branch-level inventory synchronization.
  - Provide a mobile-friendly or native mobile application for field staff.
  - Improve AI recommendations with sales seasonality, supplier lead times, and historical forecasting.

- Add cloud deployment, audit dashboards, and richer reporting exports in PDF and Excel formats.